

Linux Concepts for Embedded

Kernel / CPU / Build

- these info relates to Device Tree and driver behavior

```
uname -r: exact kernel release, used for module path matching (/lib/modules/<release>)
uname -m: CPU architecture (x86_64, aarch64, armv7l), useful for binary/driver compatibility
uname -a: full snapshot (kernel, host, build info)
```

app crashes -> CPU fault -> kernel sends **SIGSEGV** -> kernel checks core_pattern -> kernel pipes dump to systemd-coredump -> systemd-coredump stores core + metadata -> coredumpctl queries journal/storage -> coredumpctl gdb opens matching core in gdb

DebugFS

expose internal kernel and driver state for diagnosis beyond **journalctl -k** and **coredumpctl**

```
ls /sys/kernel/debug
sudo vim /sys/kernel/debug/tracing/README
```

dmesg vs persistent logs

- **dmesg** reads the kernel's ring buffer directly. Available very early in boot, but fixed size (older messages are overwritten).
- **/var/log/syslog** (or **journalctl -k**) is where a logging daemon (**rsyslogd**, **journald**) saves kernel messages to persistent disk storage. More permanent but relies on a userspace service being active.

Filesystem, Paths, and Mount Backing

- **/proc**, **/sys**: Virtual fs allocated by kernel to RAM at runtime
- **/proc** Broad runtime kernel (text name) + userspace process (process id) information. Includes per-process dirs (/proc/) and global runtime files (/proc/meminfo, /proc/interrupts).
- **/proc/sys** Writable/readable kernel tunables (sysctl interface). This is the runtime config tree for **kernel parameters**. /proc/sys/net/ipv4/ip_forward | /proc/sys/vm/swappiness
 - Inspect tunables: **sysctl -a | grep <key>**, **cat /proc/sys/...**
 - Persisted config: **/etc/sysctl.conf**, **/etc/sysctl.d/**
- **/sys** Sysfs: kernel device model and driver object hierarchy. **/sys/devices** holds real info **/sys/buses**, **/sys/classes** are softlinks giving connection view, function views **/sys/class/net/eth0** | **/sys/bus/i2c/devices** | **/sys/class/pwm/pwmchip0**
- **/dev**: Device node namespace. **devtmpfs** in RAM

- `/lib/systemd/systemd-udevd` process is in charge
- `/run`: Volatile runtime state for current boot. Usually `tmpfs` in RAM.
- `/tmp`: Temporary app workspace. Often `tmpfs` on embedded, but may also be on root filesystem.
- `mount | grep -E " on /(proc|sys|dev|run|tmp| )"'`

Block devices and mount verification

- check device id mapping

```
ls -l /dev/disk ## different view
total 0
drwxr-xr-x 3 root root 140 May 27 05:03 azure
drwxr-xr-x 2 root root 340 May 27 05:03 by-id
drwxr-xr-x 2 root root 80 May 27 05:03 by-label
drwxr-xr-x 2 root root 100 May 27 05:03 by-partuuid
drwxr-xr-x 2 root root 140 May 27 05:03 by-path
drwxr-xr-x 2 root root 80 May 27 05:03 by-uuid
# check
findmnt -no UUID,SOURCE,PARTUUID /
85a32921-52b7-4d2d-9c02-a565b70bc919 /dev/nvme0n1p1 fb505a37-7427-4a67-b014-dcf6b6c00245
# really checking the statistics
stat /dev/disk
  File: /dev/disk
  Size: 160          Blocks: 0          IO Block: 4096   directory
Device: 5h/5d   Inode: 232          Links: 8
Access: (0755/drwxr-xr-x)  Uid: (  0/   root)   Gid: (  0/   root)
Access: 2026-05-28 02:08:51.682033116 +0000
Modify: 2026-05-28 02:08:51.782033110 +0000
Change: 2026-05-28 02:08:51.782033110 +0000
 Birth: 2026-05-28 02:08:51.682033116 +0000
###
/dev/disk/by-uuid/85a32921-52b7-4d2d-9c02-a565b70bc919 -> ../../nvme0n1p1
readlink -f # resolves through the end of link chain
### check the mount options
sudo mount | grep /dev/nvme0n1
/dev/nvme0n1p1 on / type ext4 (rw,relatime,discard,errors=remount-ro)
/dev/nvme0n1p15 on /boot/efi type vfat (rw,relatime,mask=0077,dmask=0077,codepage=437,ioccharset=iso8859-1,shortname=mixed,errors=remount-ro)
# filesystem table -> what to mount right after boot
/etc/fstab
UUID=85a32921-52b7-4d2d-9c02-a565b70bc919 / ext4   discard,errors=remount-ro 0 1
```

```
# also output blkid
sudo blkid
/dev/nvme0n1p1: LABEL="cloudimg-rootfs" UUID="85a32921-52b7-4d2d-9c02-a565b70bc919" BLOCK_SIZE="4096"
TYPE="ext4" PARTUUID="fb505a37-7427-4a67-b014-dcf6b6c00245"
/dev/nvme0n1p15: LABEL_FATBOOT="UEFI" LABEL="UEFI" UUID="EBD8-3689" BLOCK_SIZE="512" TYPE="vfat"
PARTUUID="de2daaf2-b55f-4aca-ac16-18549ec4eceb"
```

```
/dev/loop1: TYPE="squashfs"
/dev/loop4: TYPE="squashfs"
# ....
/dev/nvme0n1p14: PARTUUID="c92e0d1b-d7ec-4a3b-8da2-2597e81e4eef"
```

Mount/fstab failure symptoms

- Value in `/proc/cmdline` does not exist in `blkid` output
- Broken symlink under `/dev/disk/by-uuid` or `/dev/disk/by-partuuid`
- Root mounted from unexpected device
- Boot falls into emergency shell or mounts root read-only

Disk I/O

Collect latency and throughput metrics to distinguish queueing, media, and workload bottlenecks.

```
iostat -x 1, fio --name=randrw ...
```

- FS path(s): `/sys/block/`, `/proc/diskstats`

Firmware update integrity

Verify downloaded artifact integrity before flashing to prevent bricking or partial updates.

- `sha256sum firmware.bin`
- FS path(s): `/tmp/`, `/opt/firmware/`

Boot Process

1. Boot ROM -> bootloader
 - Hardware reset path enters ROM, then first/second stage loader.
2. Bootloader phase
 - Bootloader (for example U-Boot) initializes minimum hardware, loads `kernel`, `DTB`, and optional `initramfs`.
 - `DTB` (Device Tree Blob) describes board hardware like CPU, RAM, buses, IRQ, devices.
 - `initramfs` is an early userspace filesystem in RAM for preparation tasks.
 - Bootloader sets bootargs and jumps to kernel entry.
3. Init phase (kernel early init)
 - Kernel initializes core subsystems: process/task model, virtual memory manager, interrupt handling, scheduler, VFS, and driver model.
 - `flash` and `AHB` are hardware/platform components; kernel accesses them through controllers and drivers.
4. Driver probe phase
 - Kernel buses enumerate devices and run probe functions to bind matching drivers.
5. Mount phase
 - Kernel mounts real root filesystem, then runtime mounts such as `/proc`, `/sys`, `/dev`, `/run`, `/tmp`, and optionally `/boot`, `/var`, `/home`, `/data`.
 - Common issues: wrong `root=/rootfstype=`, bad `fstab` options, missing device, read-only fallback.
6. Userspace handoff phase

- Kernel executes **PID 1** (**systemd/init/busybox init**) and userspace startup begins.
- Useful checks by stage:
 - Boot args: **cat /proc/cmdline**
 - Kernel logs: **dmesg -T | less**
 - Live kernel logs: **dmesg -w**
 - PID 1 identity: **cat /proc/1/comm**
 - Early userspace status: **systemctl --failed**

Device does not boot past kernel logo

Check kernel ring buffer to identify where boot stops (bootloader handoff, init, probe, mount, or userspace start).

```
dmesg -T
cat /proc/cmdline # shows the boot argument while
cat /var/log # may be empty if userspace passing is incomplete
```

Application binary fails with "No such file or directory"

Verify binary architecture and dynamic linker dependencies – this error often indicates missing loader/lib, not missing file.

```
g++ test.cpp -o test
g++ test.cpp -static -o test_static

file test
test: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-
linux-x86-64.so.2, BuildID[sha1]=4c9940ac6379f4bcb95cb62f9cdd04d077117316, for GNU/Linux 3.2.0, not
stripped

file test_static
test_static: ELF 64-bit LSB executable, x86-64, version 1 (GNU/Linux), statically linked,
BuildID[sha1]=83d4d3437f2e4c25a3b8e1535e4465b3cf8e4e13, for GNU/Linux 3.2.0, not stripped

ldd test
linux-vdso.so.1 (0x00007ffc58d16000)
libstdc++.so.6 => /lib/x86_64-linux-gnu/libstdc++.so.6 (0x0000744cc2a00000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x0000744cc2600000)
libm.so.6 => /lib/x86_64-linux-gnu/libm.so.6 (0x0000744cc2d30000)
/lib64/ld-linux-x86-64.so.2 (0x0000744cc2e24000)
libgcc_s.so.1 => /lib/x86_64-linux-gnu/libgcc_s.so.1 (0x0000744cc2d10000)
```

Device & Driver Model

Kernel module not loaded for peripheral

- **Device in DT:** A hardware description. The Device Tree tells the kernel "an I2C device with this name exists at this address."
- **Kernel Module:** The driver code. It tells the kernel "I am a driver that knows how to handle devices with that name." The kernel matches the DT device to the module to bind them.
- **Systemd Unit:** A userspace service manager file. It might depend on the hardware being ready (e.g., `After=dev-i2c-1.device`) but does not interact with the kernel driver model directly. Module uses expected metadata for autoloading.

```
lsmod, modprobe <module>, modinfo <module> /lib/modules/$(uname -r)/, /etc/modules-load.d/ [Q]
```

Compare kernel mod, systemd unit and device in DT

Device-tree change not taking effect

Verify active DT blob content and boot logs to confirm your updated DTB/overlay is actually used.

- **hexdump:** A general-purpose tool to display any file's content in hexadecimal, octal, or ASCII format. Used here to view raw binary content of the DTB.
- **coredump:** Not a command, but a file containing a snapshot of a crashed process's memory and state. Generated by the kernel for post-mortem debugging with `gdb`.

```
hexdump /sys/firmware/fdt, dmesg | grep -i dt /boot/, /sys/firmware/devicetree/base/ [Q]
```

hexdump and coredump?

```
cat /proc/cmdline
BOOT_IMAGE=/boot/vmlinuz-6.8.0-1052-azure root=PARTUUID=fb505a37-7427-4a67-b014-dcf6c00245 ro
console=tty1 console=ttyS0 earlyprintk=ttyS0 nvme_core.io_timeout=240 panic=-1
```

USB / I2C / SPI sensor not detected

Check bus enumeration and kernel probe messages to confirm whether the device is electrically visible and driver-bound. `lsusb, dmesg | grep -i usb /sys/bus/usb/devices/ /sys/bus/i2c/devices/ /sys/bus/spi/devices/`

GPIO line cannot be toggled

Identify line ownership and direction; contention by another driver/process is a common cause.

```
gpioinfo, gpioset gpiochip0 17=1 /dev/gpiochip*, /sys/kernel/debug/gpio
```

Serial port not receiving data

Validate baud, parity, stop bits, and raw/canonical mode before assuming hardware failure. `stty -F /dev/ttyS0 -a, cat /dev/ttyS0 /dev/ttyS*, /sys/class/tty/`

udev, hotplug, and device permissions

Hotplug is the act of adding or removing devices while the system is running. The **udev device event pipeline** handles this:

1. Kernel detects a hardware change (e.g., USB device plugged in) and creates a "uevent".
2. `udev` (the userspace daemon) receives the uevent.
3. `udev` processes its rules (`/etc/udev/rules.d/`) to match the event's properties.

4. On a match, it performs actions like creating a device node (`/dev/sdb1`), setting permissions, and loading a driver.

`udev` also sets permissions (owner, group, mode) on device nodes in `/dev` when they are created. Rules in `/etc/udev/rules.d/` can match device attributes (like vendor/product ID) and specify `OWNER`, `GROUP`, and `MODE`.

Troubleshooting hotplug: `udevadm monitor`, `udevadm test /sys/class/...` Checking device permissions: `ls -l /dev/<node>`, `id`, `getfacl /dev/<node>`

- `ls -l` and `getfacl` check file-level permissions but don't show mount options. Mount options like `ro` are enforced by the VFS layer and override file-level permissions.

`/etc/udev/rules.d/`, `/run/udev/` [Q] Is mount option checked by these commands as well

Networking

Network interface is down after boot

Check link state and manually bring interface up to differentiate config issue from driver or PHY issue. `ip link show`, `ip link set eth0 up`, `ip netns help` The `ip link` command operates at Layer 2 (Data Link). It manages network interface devices themselves (MAC addresses, state up/down), not higher-level protocols like IP (Layer 3) or TCP/UDP (Layer 4). `/sys/class/net/`, `/etc/network/`

Embedded board cannot get IP via DHCP

Run DHCP client in verbose mode to inspect discover/offer/request/ack flow and lease failures. `udhcpc -i eth0 -v`, `ip addr /etc/resolv.conf`, `/var/lib/` A DHCP client (like `dhclient` or `systemd-networkd`) can update `/etc/resolv.conf` with the DNS server addresses provided in the DHCP lease. However, on modern systems using `systemd-resolved`, this file may just point to the local resolver stub (`nameserver 127.0.0.53`).

DNS resolution fails but ping by IP works

Validate resolver configuration and query path to separate network reachability from name service problems. `cat /etc/resolv.conf`, `nslookup example.com /etc/nsswitch.conf` (Name Service Switch) defines the order for name resolution. The `hosts:` line (e.g., `hosts: files dns`) tells the system to first check `/etc/hosts` (`files`) and then query DNS servers (`dns`).

NTP time sync not working

Confirm local time service state and peer synchronization to prevent timestamp and TLS issues. `timedatectl status`, `ntpq -p /etc/systemd/timesyncd.conf`, `/var/lib/systemd/timesync/`

RTC time drifts after power cycle

Compare hardware clock and system clock behavior across reboot to find backup-power or sync process issues. `hwclock -r`, `hwclock -w /dev/rtc0`, `/sys/class/rtc/rtc0/`

- **RTC (Real-Time Clock):** A battery-backed hardware clock that keeps time even when the system is powered off.

- **System Clock:** The main software clock kept by the kernel since boot. More precise but lost on reboot.
- **NTP (Network Time Protocol):** A protocol to synchronize the system clock with accurate time servers over the network. The usual flow is: RTC sets system clock at boot → NTP corrects system clock → system clock updates RTC on shutdown.

Suspect network packet loss in field

Combine ICMP tests, NIC counters, and packet capture to localize loss on host, link, or upstream path.

`ping -c 50 <ip>, ethtool -S eth0, tcpdump -i eth0 /sys/class/net/eth0/statistics/`

Systemd: Units, Cgroups, and Namespaces

- **systemd:** Main service manager and init system with PID==1. Parent of most processes.
- **systemd-coredump:** Started only when a unit crashes. `coredumpctl` for debug later. A unit enters **failed** state if its main process terminates with non-zero exit code, unhandled signal (e.g., SIGSEGV), or timeout.
- **journalctl:** Records kernel `-k`, boot `-b`, and unit logs.

Enablement

- Whether the unit starts at boot
- `systemctl enable / disable / is-enabled myapp.service`

Activation

- Start / stop the unit right now
- `systemctl start / stop / status myapp.service`

Status

- `running exited reloading failed`
- `systemctl reset-failed my-unit.service → inactive`

Units

`.service, .socket, .mount, .target`

CgroupV2

Controls what a process can **USE**

```
sudo systemd-cgls
Control group /:
-.slice
├user.slice
│  └user-1000.slice
│     └user@1000.service ...
│        └user.slice
│           └podman-pause-9141885347830502205.scope
│              └3199 /usr/bin/podman
```

```
| | |─app.slice
| | |  └─dbus.service
| | |    └─3419 /usr/bin/dbus-daemon --session --address=systemd>
```

- **v2** redesigns over **v1** as resources **cpu io mem** can only be at leaf process
- for units **.service .scope** systemd creates a cgroup
- cgroup tree enables kill all offsprings when a node is killed
- Can pass resource management flag like **CPUWeight** at unit files

Namespace

Controls what a process can **SEE**

- **systemd** can apply per-unit namespace isolation (mount, IPC, PID, network, user, UTS).
- It does not globally "own" namespaces; kernel provides namespaces, systemd configures usage per unit.
- **systemctl show <unit> | grep -E "Namespace|Private|Protect"**

```
sudo lsns

      NS TYPE   NPROCS   PID USER      COMMAND
4026531834 time       162     1 root      /sbin/init
4026531835 cgroup     162     1 root      /sbin/init
4026531836 pid       162     1 root      /sbin/init
4026531837 user      161     1 root      /sbin/init
4026531838 uts        158     1 root      /sbin/init
4026531839 ipc        162     1 root      /sbin/init
4026531840 net         162     1 root      /sbin/init
```

Process crashes

Correlate service logs with core dump records to identify recurring crash signatures. **journalctl -u app.service -n 200, coredumpctl list /var/log/journal/, /var/lib/systemd/coredump/**

- coredump will not start until some failure

High CPU usage causes missed control loop deadlines

Find top CPU consumers and scheduling priorities to diagnose realtime jitter and starvation. **top, ps -eo pid,comm,rtprio,ni,%cpu --sort=-%cpu rtprio** is the real-time priority (1-99). Higher numbers are higher priority. A value of **50** is a common default for **SCHED_RR** or **SCHED_FIFO** policies, while **99** is the maximum. A **-** or **0** indicates the process is using the normal **SCHED_OTHER** scheduler, not a real-time one. **/proc/, /sys/fs/cgroup/**

Need startup order dependency fix

Inspect merged unit files and dependency tree to enforce correct sequencing. **systemctl cat app.service, systemctl list-dependencies /etc/systemd/system/*.service**

Memory Management

Out-of-memory kills critical process

- Resident Set Size (RSS) is the portion of a process's memory held in physical RAM. Other types include Virtual Memory Size (VMS) and Proportional Set Size (PSS).
- The kernel's OOM killer calculates a "badness" score for each process (`oom_score`). The score is primarily based on the percentage of memory the process is using. Processes with higher scores are killed first. The score can be adjusted via `/proc/<pid>/oom_score_adj`.

`dmesg | grep -i oom, cat /proc/<pid>/oom_score /proc/sys/vm/, /var/log/` [Q] memory type like `rsmem ....` [Q] oomkill score calculator

Memory leak suspected in long run

Track resident memory trends and process mappings over time to confirm progressive growth. `free -m, cat /proc/<pid>/status`

- **MemTotal:** Total usable RAM.
- **MemFree:** Unused RAM.
- **MemAvailable:** An estimate of how much memory is available for starting new applications, without swapping. It accounts for reclaimable cache and buffers.
- **Buffers:** Memory used by kernel buffers (e.g., for block device I/O).
- **Cached:** Memory used for the page cache (caching file contents).
- **SReclaimable:** The part of the slab cache that can be reclaimed under memory pressure (e.g., caches for dentries and inodes).
- **SUnreclaim:** The part of the slab cache that is not reclaimable.
- **Shmem:** Memory used by shared memory (`tmpfs`).
- **Active:** Memory that has been used more recently and is usually not reclaimed before Inactive memory.
- **Inactive:** Memory that has been used less recently and is a better candidate for reclamation.

Memory leaks in C/C++ are most often caused by failing to release dynamically allocated memory (`new`, `malloc`) when it's no longer needed, often due to missing or incorrect `delete/free` calls in destructors or error paths. [Q] Is memory leak often caused by faulty or lack-of destructor for data

Process

`ps -p 194 -o pid,ppid,user,comm,args`

User Management, Permissions, and Privilege Escalation

- Subject `uid`, `gid`, `process`
- Object `device`, `inode`
- DAC Discretionary Access Control for file `uid`, `gid` and supplementary group mode bits: `dr-xr-xr-x` == `[type][owner user][owner group][other]` ACL extends this by file `getfacl` `setuid/setgid` turns process credential from caller `uid/gid` to owner `uid/gid` sticky bits sticky: on shared directories, only file owner/root can delete or rename entries

- Linux capabilities `grep Cap /proc/$$/status` getcap: capabilities attached to executable files (xattr security.capability) `/proc//status` (CapEff, CapPrm, CapInh, etc): capabilities currently held by a process `capsh --print`: process capability sets in current shell context `CAP_NET_ADMIN | CAP_SYS_ADMIN` can only assign to process
- MAC permissions : SELinux, AppArmor policy-based access control SELinux: `getenforce`, `sestatus`, `ls -Z <path>`, `ps -eZ` AppArmor: `aa-status`, `cat /sys/module/apparmor/parameters/enabled`, `journalctl -k | grep -i apparmor`
- Mount options `ro` blocks writes even if file permissions would allow writing. `noexec` blocks executing binaries from that mount even if x bit is set.
- Device policy: `udev` assigns ownership/mode/group on device nodes under `/dev`. this is DAC permission setup for device-node objects (uid/gid/mode bits, and optionally ACL via udev rules).
- `id uid=1000(hsunwen) gid=1000(hsunwen) groups=1000(hsunwen),4(adm),20(dialout),24(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(video),46(plugdev),119(netdev),120(lxd)`
- `groups <user>`
- `ls -l /dev/<node>`
- `getcap -r / 2>/dev/null | head`

Privilege escalation

- `sudo` (command-level delegation)
 - Runs a specific command as another user (default: root).
 - Policy is defined in `/etc/sudoers` and `/etc/sudoers.d/`.
 - Users authenticate as themselves, then are authorized by rule.
 - Best for day-to-day ops because privilege is narrow and explicit.
- `su -` (full identity switch)
 - Switches to another account (often root) with a login shell environment.
 - `-` loads target user's profile and environment (`HOME`, `PATH`, shell init files).
 - poor auditing
- `polkit` (action-based authorization for system services)
 - Authorizes privileged actions exposed by system daemons (often over D-Bus).
 - Common with desktop and service management workflows (for example NetworkManager, udisks, package frontends).
- `sudo -l` : show allowed commands for current user.
- `sudo -v` : refresh sudo credential timestamp.
- `visudo -c` : syntax-check sudoers safely.
- `journalctl _COMM=sudo` or distro auth logs: inspect sudo activity.
- `pkaction` : list polkit actions.

- `pkcheck --action-id <action> --process $$` : test polkit authorization for current process.
- Embedded recommendation
 - Prefer `sudo` for operational commands and automation.
 - Keep root login tightly controlled.
 - Use `su -` only when an interactive full root environment is truly required.